



UNIVERSIDADE ESTADUAL DO CEARÁ
CENTRO DE CIÊNCIAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
DOUTORADO EM CIÊNCIA DA COMPUTAÇÃO

ANTÔNIO SÉRGIO DE SOUSA VIEIRA

MODELO DE DECISÃO BASEADO EM LÓGICA FUZZY E APRENDIZADO POR
REFORÇO PROFUNDO PARA O OFFLOADING DE TAREFAS EM REDES
VEICULARES 5G

FORTALEZA – CEARÁ

2026

ANTÔNIO SÉRGIO DE SOUSA VIEIRA

MODELO DE DECISÃO BASEADO EM LÓGICA FUZZY E APRENDIZADO POR
REFORÇO PROFUNDO PARA O OFFLOADING DE TAREFAS EM REDES VEICULARES
5G

Proposta de qualificação apresentada ao Curso de Doutorado em Ciência da Computação do Programa de Pós-Graduação em Ciência da Computação do Centro de Ciências e Tecnologia da Universidade Estadual do Ceará, como requisito parcial para a defesa de tese.
Orientador: Dr. Joaquim Celestino Júnior

FORTALEZA – CEARÁ

2026

ANTÔNIO SÉRGIO DE SOUSA VIEIRA

MODELO DE DECISÃO BASEADO EM LÓGICA FUZZY E APRENDIZADO POR
REFORÇO PROFUNDO PARA O OFFLOADING DE TAREFAS EM REDES VEICULARES
5G

Proposta de qualificação apresentada ao Curso de Doutorado em Ciência da Computação do Programa de Pós-Graduação em Ciência da Computação do Centro de Ciências e Tecnologia da Universidade Estadual do Ceará, como requisito parcial para a defesa de tese.

Aprovada em:

BANCA EXAMINADORA

Dr. Joaquim Celestino Júnior (Orientador)
Universidade Estadual do Ceará – UECE

Membro da Banca Dois
Universidade do Membro da Banca Dois -
SIGLA

Membro da Banca Três
Universidade do Membro da Banca Três -
SIGLA

Membro da Banca Quatro
Universidade do Membro da Banca Quatro -
SIGLA

Membro da Banca Cinco
Universidade do Membro da Banca Cinco -
SIGLA

Membro da Banca Seis
Universidade do Membro da Banca Seis -
SIGLA

RESUMO

O crescimento exponencial de dispositivos conectados e a emergência das redes veiculares 5G (V2X) têm impulsionado a necessidade de soluções eficientes de processamento distribuído, especialmente em cenários de *vehicular edge computing* (VEC). O desafio central reside em como decidir, de forma dinâmica e adaptativa, quando e onde descarregar (*offload*) tarefas computacionais. Este trabalho propõe um modelo híbrido que integra Lógica Fuzzy e Aprendizado por Reforço (Reinforcement Learning – RL) para otimizar o processo de *task offloading* em ambientes veiculares 5G. A Lógica Fuzzy é utilizada para modelar e quantificar as incertezas inerentes ao ambiente veicular (como qualidade do link, mobilidade e carga dos servidores de borda), transformando-as em indicadores de estado robustos. Esses indicadores são então utilizados como entrada para um agente de DRL, que aprende políticas adaptativas de alocação de tarefas visando maximizar uma função de recompensa multiobjetivo, focada em minimizar a latência fim-a-fim e o consumo energético. Os resultados esperados incluem um aprendizado mais estável e uma política de descarregamento mais eficiente em cenários densos e altamente móveis.

Palavras-chave: task offloading; reinforcement learning; fuzzy topsis; 5G; Rede Veicular.

ABSTRACT

In this on the radio waves: popular culture, peasants and the Basic Education Movement we analyze the participation of peasants of the Brazilian northeastern region in the Basic Education Movement. The focus of this thesis is to demonstrate how the labors involved with broadcast schools have elaborated actions for maintaining and spreading the schools in their communities, in order to achieve the necessary means to improve their way of life. Peasants of the Basic Education Movement have been coadjuvant of the modernizing catholic proposition of the early 1960s, by means of quite peculiar political and cultural representations. Some of these representations were: a meaning for the school, a role for the union and for the political participation, precepts of the land use rights and labor rights, and the multiple meanings of the radio as a mass communication, information and leisure medium. This study intends to stress that the actions – and the political enrollment – of the northeastern peasant could not ever be separated from the modernizing process. The connection can be observed in different social movements of the period, such as the Basic Education Movement, rural unions, the Catholic Agrarian Youth and the MCP. In this sense, we consider that, if the Brazilian modernization was a guideline for the institutions, political organisms and parties for the social movement, such a modernization was a guideline of demands based on elements of material life. Those elements included, by that time, the agrarian reform, the educational issue and labor urgencies.

Keywords: adult education; community schools; peasants; popular culture.

LISTA DE ILUSTRAÇÕES

Figura 1 – Formas de descarregamento de tarefas em VEC: <i>full</i> , <i>partial</i> e <i>adaptive offloading</i> , com seus trade-offs de confiabilidade e complexidade.	13
Figura 2 – Principais problemas inerentes ao <i>task offloading</i> veicular: mobilidade e instabilidade do enlace, heterogeneidade de recursos e filas, custos energéticos, incerteza de medições, dependência de tecnologias/protocolos e complexidade <i>NP-hard</i> sob restrições de latência e confiabilidade.	15
Figura 3 – O gráfico ilustra a colinearidade perfeita: todos os pontos alinham-se em uma reta, indicando que uma variável é redundante em relação à outra.	21
Figura 4 – Exemplo de trajetória de regimes $Z(t)$ ao longo do tempo (sinal em degraus). Cada patamar representa um tempo de permanência no regime; as transições representam mudanças de fase (Markov-modulado).	23
Figura 5 – Mesmo regime da Figura 4, agora mapeado para um alvo $u(t)$ (degraus) e para uma variável observável $y(t)$ que responde com inércia de primeira ordem (rampas suaves).	24
Figura 6 – Regressão linear simples: Dados observados (pontos) e reta de regressão ($y \approx x + 1$) ajustada pelo modelo supervisionado.	30
Figura 7 – Conceito de agrupamento (clustering): identificação de estruturas latentes em dados não rotulados.	31
Figura 8 – Ciclo de interação agente-ambiente no aprendizado por reforço: o agente observa o estado do ambiente, seleciona uma ação segundo sua política aprendida, executa a ação e recebe uma recompensa (consequência), iniciando um novo ciclo.	31
Figura 9 – Distribuições de probabilidade de recompensa para um problema de 4-braços ($k = 4$). Cada curva representa a densidade de probabilidade da recompensa para uma ação específica, ilustrando diferentes médias (μ) e desvios padrão (σ).	32
Figura 10 – Recompensa acumulada utilizando a estratégia gananciosa (Greedy).	33
Figura 11 – Comparação de desempenho: Agente Ganancioso (Greedy) vs. Agente ε -Ganancioso (ε -Greedy).	34

Figura 12 – Exemplo de um problema k-armed bandit não-estacionário: as médias das recompensas ($q_*(a)$) variam ao longo do tempo, caracterizando diferentes regimes onde ações distintas se tornam ótimas. Note como a Ação 3 (verde) inicia inferior mas torna-se dominante após $t \approx 500$, enquanto a Ação 1 (azul) degrada-se.	35
Figura 13 – Comparação de estratégias em ambiente não-estacionário: Greedy vs. ϵ-Greedy com média amostral vs. ϵ-Greedy com taxa constante ($\alpha = 0.1$). A estratégia com taxa constante demonstra melhor adaptação às mudanças dinâmicas.	36
Figura 14 – MDP didático com dois estados (α, β), ações explícitas (λ, ν, μ), probabilidades de transição e recompensas associadas.	37

LISTA DE TABELAS

Tabela 1 – Correlação entre tempo de processamento e consumo energético em um cenário típico de simulação	21
Tabela 2 – Resumo dos perfis de aplicação adotados para o gerador de tarefas. O tamanho está em KB (decimal), a densidade computacional em ciclos/byte e o deadline em segundos.	27
Tabela 3 – Resumo dos perfis de CPU. A frequência é dada em GHz (frequência efetiva usada no modelo), e o número de núcleos representa a capacidade de paralelismo do recurso no simulador.	28
Tabela 4 – Resumo dos perfis de energia computacional. O perfil 0 usa um modelo dependente de frequência, enquanto o perfil 1 usa energia constante por ciclo, ambos recorrentes na literatura.	29
Tabela 5 – Exemplo de conjunto de dados supervisionado com múltiplos previsores e um alvo.	30

LISTA DE ALGORITMOS

Algoritmo 1 – Algoritmo ε -Greedy para o problema k -armed bandit	34
Algoritmo 2 – Algoritmo ε -Greedy para o problema k -armed bandit não-estacionário	36

LISTA DE CÓDIGOS-FONTES

SUMÁRIO

1	INTRODUÇÃO	11
1.1	Contextualização	11
1.2	Motivação	12
1.3	Definição do Problema	12
1.4	Questões de Pesquisa	15
1.5	Hipóteses	16
1.6	Objetivos	17
1.6.1	Objetivo Geral	17
1.6.2	Objetivos Específicos	17
1.7	Contribuições Esperadas	17
1.8	Publicação	18
1.9	Organização do Trabalho	19
2	FUNDAMENTAÇÃO TEÓRICA	20
2.0.1	Redes Veiculares	20
2.0.2	Comunicação 5G e Sidelink	20
2.0.3	Computação Veicular de Borda	20
2.0.4	Descarregamento de Tarefas	20
2.0.5	Requisitos Metodológicos para Simulação Não Estacionária em Redes Veicu- lares	20
2.0.6	Perfis de Tarefas no Contexto de Redes Veiculares	24
2.0.7	Aprendizado por Reforço	29
2.0.8	Lógica Fuzzy	37
2.0.9	Fuzzy-TOPSIS	37
2.0.10	Contexto	37
2.0.11	Considerações Finais	37
3	METODOLOGIA	38
3.1	Modelagem do Sistema e Formulação do Problema	38
GLOSSÁRIO		40
APÊNDICE A – LOREM IPSUM		41
APÊNDICE B – MODELO DE CAPA		42
APÊNDICE C – TERMO DE FIEL DEPOSITÁRIO		43

ANEXO A – EXEMPLO DE ANEXO	44
ANEXO B – DINÂMICA DAS CLASSES SOCIAIS	45

1 INTRODUÇÃO

Neste capítulo serão apresentados o contexto (Seção 1.1), a motivação (Seção 1.2), a definição do problema (Seção 1.3), as questões de pesquisa (Seção 1.4), as hipóteses (Seção 1.5), os objetivos (Seção 1.6), as contribuições esperadas (Seção 1.7), a publicação conseguida (Seção 1.8) e finalmente como será realizada a organização do trabalho (Seção 1.9).

1.1 Contextualização

A convergência entre tecnologias de comunicação, eletrificação veicular e condução autônoma tem moldado um novo paradigma para o setor de transportes. O conceito de Internet de Veículos (IoV) transforma veículos em sistemas computacionais sofisticados, capazes de viabilizar diversas aplicações de Sistemas de Transporte Inteligentes (ITS) (??).

Veículos modernos são equipados com múltiplos sensores, unidades de processamento e interfaces de comunicação, permitindo a coleta e o processamento de grandes volumes de dados. Contudo, a complexidade das aplicações e a quantidade crescente de dados a serem processados podem exceder a capacidade computacional das unidades de bordo (*On-Board Unit* - OBU) instaladas.

Uma solução imediata seria aumentar a capacidade computacional dessas OBUs, possibilitando maior processamento local. No entanto, o atual período de transição tecnológica resulta em cenários heterogêneos, no qual coexistem OBUs com diferentes capacidades computacionais (??) e distintas tecnologias de comunicação (??). Essa diversidade representa um desafio fundamental para a implementação de aplicações veiculares em larga escala, especialmente no contexto das cidades inteligentes (*Smart Cities*) (??).

Nesse ambiente fortemente dinâmico e caótico, muitas aplicações, em particular aquelas voltadas à condução autônoma, impõem requisitos rigorosos de latência (??). A execução integral do processamento nos dispositivos terminais (*User Equipments* – UEs) frequentemente inviabiliza o atendimento a tais requisitos, devido às limitações físicas de energia e capacidade computacional dos chips embarcados (??).

Para superar essas restrições, destaca-se a Computação Veicular de Borda (*Vehicular Edge Computing* – VEC), um paradigma onde servidores computacionalmente robustos são posicionados próximos aos veículos, reduzindo de forma significativa a latência de processamento (??). Tecnologias relacionadas, como a Computação em Nuvem Veicular (*Vehicular Cloud Computing* – VCC) (??) e a Computação em Névoa Veicular (*Vehicular Fog Computing* –

VFC) (??), ampliam o escopo do processamento distribuído, oferecendo suporte adicional em diferentes níveis de proximidade e escalabilidade.

O descarregamento de tarefas (*task offloading*) surge como o mecanismo central para operacionalizar a comunicação e o processamento colaborativo entre veículos e essas arquiteturas distribuídas. Essa técnica consiste em migrar tarefas computacionais para nós mais capazes, como servidores de borda ou outros veículos com maior poder de processamento (??). Dessa forma, veículos com recursos limitados conseguem executar aplicações complexas, aproveitando melhor a infraestrutura distribuída disponível na rede.

O principal desafio, entretanto, está em decidir quando e para onde descarregar as tarefas, considerando múltiplos fatores conflitantes, como tempo limite de processamento, custo energético, balanceamento de carga e qualidade do enlace de comunicação. Abordagens determinísticas, baseadas exclusivamente em heurísticas, apresentam desempenho limitado diante da alta variabilidade espacial e temporal do ambiente veicular (??). Mobilidade intensa, flutuações do canal e mudanças abruptas na carga computacional tornam as decisões estáticas rapidamente obsoletas, comprometendo a eficiência do sistema.

Essa limitação tem impulsionado a busca por soluções adaptativas e mais robustas, baseadas em aprendizado de máquina e sistemas inteligentes (????????????). Diante desse cenário, este trabalho propõe um modelo híbrido para o descarregamento de tarefas que integra técnicas de aprendizado de máquina (ML) e lógica fuzzy. Essa abordagem combina a capacidade do ML de capturar padrões dinâmicos e realizar decisões adaptativas com a habilidade da lógica fuzzy de lidar com incertezas e valores imprecisos do ambiente veicular, oferecendo um mecanismo mais flexível e eficaz para tomada de decisão em tempo real.

1.2 Motivação

1.3 Definição do Problema

Diante das limitações de processamento local e da elevada variabilidade do ambiente veicular, o problema central investigado neste trabalho pode ser formulado da seguinte maneira:

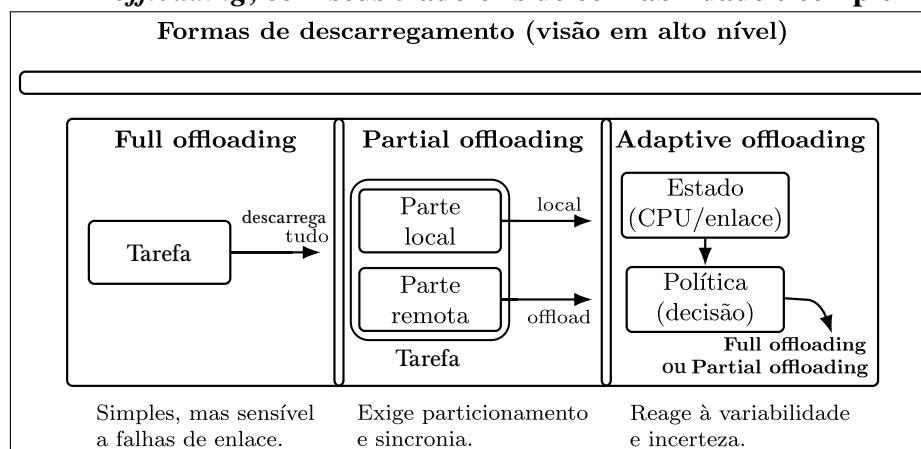
Como realizar o descarregamento de tarefas em sistemas de Computação Veicular de Borda (VEC) ou em outros dispositivos veiculares de modo confiável e adaptativo, garantindo redução no tempo de execução, diminuição do consumo energético e adequado balanceamento de carga entre os nós disponíveis?

Estudos recentes reforçam que a alta mobilidade, a intermitência dos enlaces e a heterogeneidade de recursos tornam esse desafio ainda mais complexo (????????).

Para delimitar o escopo da solução proposta, é importante destacar que o problema de task offloading, em sua essência, configura-se como um problema de otimização *NP-hard* (????). Isso implica que soluções exatas em tempo polinomial são inviáveis para cenários dinâmicos como o veicular, o que direciona a busca por métodos de baixa complexidade computacional e boa capacidade de adaptação.

Outro aspecto importante refere-se a maneira na qual o descarregamento é realizado, podendo ser classificado em três categorias principais: *full offloading* (descarregamento total da tarefa), *partial offloading* (descarregamento parcial) e *adaptive offloading* (descarregamento adaptativo) (??) como pode ser observado na Figura ???. Cada abordagem apresenta vantagens e desvantagens específicas, influenciando diretamente a formulação do problema e as estratégias de solução adotadas.

Figura 1 – Formas de descarregamento de tarefas em VEC: *full*, *partial* e *adaptive offloading*, com seus trade-offs de confiabilidade e complexidade.



Fonte: Elaborado pelo autor

Nesse contexto, a literatura apresenta diversas abordagens para modelar e melhorar a tomada de decisão de descarregamento de tarefas, por exemplo: programação linear e inteira para cenários de Computação de de Borda Multiacesso (*Multi-access Edge Computing* - MEC) (??), meta-heurísticas como algoritmos genéticos (??), PSO (??) e colônia de formigas (??) para problemas de grande escala, além de abordagens baseadas em aprendizado de máquina e aprendizado por reforço profundo, adequadas para ambientes altamente variáveis e parcialmente observáveis (??????).

Outra consideração fundamental diz respeito à modelagem do ambiente veicular. Cada solução proposta na literatura assume combinações específicas de tecnologias de comunica-

ção, capacidades computacionais e modelos de consumo energético, o que impacta diretamente a forma como o problema é otimizado (??). Além disso, deve-se definir se o descarregamento será realizado apenas para servidores de borda, para veículos vizinhos (*offloading* colaborativo) ou para ambos, como no paradigma de *Vehicular Fog Computing (VFC)* (??). Essa escolha é influenciada pela disponibilidade de recursos, pelo perfil da aplicação e pela estabilidade dos enlaces, variando conforme o cenário de estudo.

A mobilidade veicular constitui outro elemento crítico. Diferentes cenários, como os urbanos, suburbanos e rodoviários, apresentam padrões distintos de flutuação do canal, densidade de veículos e comportamento do tráfego. Em áreas urbanas, fatores como congestionamentos, presença de pedestres, ônibus e motocicletas podem alterar significativamente a dinâmica da conectividade e, conseqüentemente, as estratégias de *offloading* (????). Tais aspectos precisam ser considerados para que a tomada de decisão seja realista e eficaz.

A viabilização do processo de decisão depende de mecanismos eficientes de comunicação e descoberta de recursos. Neste trabalho, assume-se comunicação via 5G NR (*Fifth-Generation Wireless Technology NewRadio*) entre os veículos e os servidores de borda, e comunicação V2X (*Vehicle-to-Everything*) entre os veículos, alinhado às diretrizes das arquiteturas 3GPP Release 16/17 (??). Além disso, protocolos de descoberta e anúncios de recursos são necessários para que os veículos obtenham informações sobre capacidade de CPU, características relacionadas ao estado energético e qualidade do enlace dos dispositivos vizinhos (????).

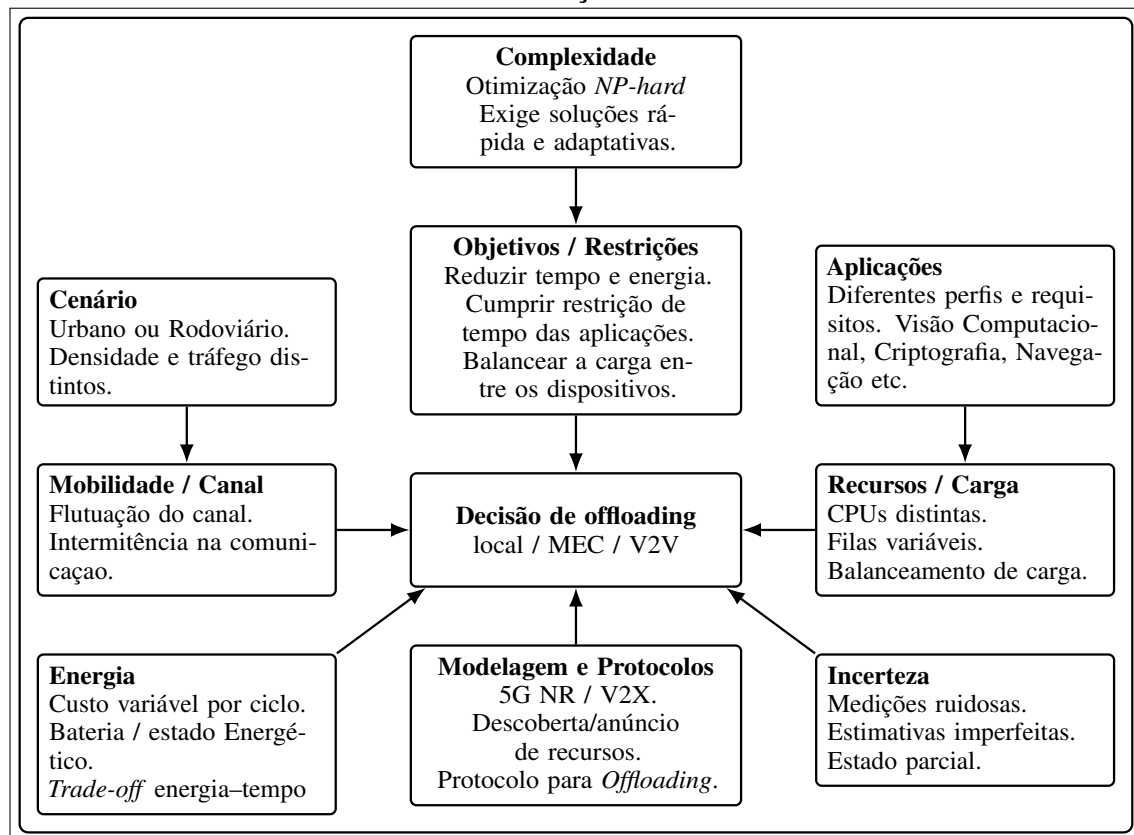
Outro ponto essencial refere-se ao perfil das aplicações beneficiadas pelo descarregamento. A popularização de hardware embarcado mais poderoso em OBUs permite o processamento local de aplicações leves; contudo, aplicações com baixa carga de dados, mas com alta demanda computacional, como visão computacional, detecção de objetos, fusão sensorial, criptografia e algoritmos de navegação autônoma, são candidatas naturais ao *offloading* (????). Para evitar latências excessivas, é necessário equilibrar o custo de transmissão com o ganho de processamento externo, preservando as restrições temporais impostas pela aplicação.

Somente após a coleta e integração dessas informações é possível realizar a tomada de decisão sobre o descarregamento. Essa decisão deve considerar a incerteza inerente às medições, a rápida variação do ambiente e a necessidade de minimizar falhas. Assim, soluções adaptativas baseadas em aprendizado de máquina, reforço profundo ou lógica fuzzy têm sido exploradas como alternativas promissoras para lidar com a incerteza, o dinamismo e a complexidade dos cenários veiculares (??????).

De forma geral pode-se resumir os problemas enfrentados através da Figura 2 onde

uma decisão de descarregamento de tarefa deve considerar o cenário observado, as tecnologias de comunicação empregadas, as aplicações utilizadas, a incerteza do ambiente, o custo e o objetivo buscado.

Figura 2 – Principais problemas inerentes ao *task offloading* veicular: mobilidade e instabilidade do enlace, heterogeneidade de recursos e filas, custos energéticos, incerteza de medições, dependência de tecnologias/protocolos e complexidade *NP-hard* sob restrições de latência e confiabilidade.



Fonte: Elaborado pelo autor

1.4 Questões de Pesquisa

Este trabalho busca responder às seguintes questões de pesquisa, formuladas a partir das limitações inerentes ao ambiente veicular, da complexidade do problema de descarregamento de tarefas e da necessidade de garantir confiabilidade, eficiência energética e baixa latência em sistemas VEC/V2X:

QP1. Como realizar o descarregamento de tarefas em ambientes VEC/V2X de maneira confiável e adaptativa, considerando a alta variabilidade da mobilidade veicular, do canal de comunicação e da carga computacional?

QP2. De que forma modelos híbridos, combinando aprendizado de máquina e

lógica fuzzy, podem melhorar a tomada de decisão de *task offloading* em cenários veiculares dinâmicos, mitigando incertezas na coleta de informação e na predição do estado futuro da rede?

QP3. Quais são os impactos do uso conjunto de comunicação 5G NR e V2X no desempenho do *task offloading*, especialmente na redução da latência, no consumo energético e na taxa de falhas de descarregamento?

QP4. Como diferentes perfis de aplicação veicular, especialmente aquelas com baixa carga de dados e alta demanda computacional, influenciam a decisão de quando, para onde e quanto descarregar, considerando limites de latência, balanceamento de carga e consumo energético?

1.5 Hipóteses

A partir das questões acima e da fundamentação teórica apresentada, formulam-se as seguintes hipóteses de pesquisa:

- H1.** A integração de aprendizado de máquina com lógica fuzzy é capaz de produzir um modelo de decisão adaptativo que supera métodos puramente determinísticos ou heurísticos em cenários VEC/V2X altamente dinâmicos.
- H2.** O uso de modelos híbridos reduz o impacto das incertezas inerentes à mobilidade veicular, permitindo estimar de forma mais robusta o melhor destino para o descarregamento de tarefas.
- H3.** A utilização conjunta de 5G NR e V2X proporciona atendimento as requisitos das aplicações mais estáveis e de menor latência, aumentando a confiabilidade do *offloading* e reduzindo a probabilidade de falhas decorrentes de desconexões súbitas.
- H4.** Aplicações com baixa carga de dados e alta demanda computacional apresentam o maior ganho com o *offloading*, desde que o modelo consiga estimar adequadamente o custo energético e o impacto na latência de processamento.

1.6 Objetivos

1.6.1 Objetivo Geral

Desenvolver e avaliar um modelo híbrido de decisão, baseado na integração da lógica fuzzy com o aprendizado por reforço, para otimizar o descarregamento de tarefas multiobjetivo em redes veiculares 5G NR (VEC/V2X).

1.6.2 Objetivos Específicos

- OE1. Realizar uma revisão sistemática de algoritmos de *task offloading* em cenários VEC, focando em abordagens de DRL e Lógica Fuzzy.**
- OE2. Modelar as incertezas do ambiente veicular (ex: qualidade do link, carga do servidor, mobilidade) usando Lógica Fuzzy, projetando funções de pertinência e regras de inferência para gerar indicadores de estado robustos.**
- OE3. Desenvolver um agente Híbrido Fuzzy-RL que utilize os indicadores fuzzy gerados como parte de seu vetor de estado, e que aprenda uma política de *offloading* para otimizar uma função de recompensa que equilibre latência, balanceamento de carga, taxa de sucesso no *offloading* e consumo energético.**
- OE4. Validar a arquitetura híbrida proposta em ambiente de simulação (NS-3), comparando seu desempenho (latência fim-a-fim, consumo de energia e taxa de sucesso no *offloading*) com algoritmos de DRL puros (não-fuzzy) e heurísticas tradicionais.**

1.7 Contribuições Esperadas

Espera-se que este trabalho produza contribuições relevantes tanto do ponto de vista científico quanto do ponto de vista prático, sobretudo no contexto de sistemas veiculares inteligentes e computação de borda. As principais contribuições previstas são:

- CE1. Proposição de um modelo híbrido para descarregamento de tarefas em ambientes veiculares, integrando técnicas de aprendizado de máquina com lógica fuzzy. O modelo pretende oferecer uma tomada de decisão mais robusta e adaptativa do que abordagens puramente determinísticas ou exclusivamente baseadas em ML, sobretudo em cenários com forte incerteza e alta mobilidade.**

- CE2.** Modelagem detalhada do ambiente de comunicação e processamento veicular, incluindo enlaces 5G NR (com alinhamento às diretrizes do 3GPP Release 17), comunicação V2X sidelink, parâmetros de mobilidade e heterogeneidade computacional entre veículos e servidores de borda. Essa modelagem fornece uma plataforma de simulação realista para decisões de offloading.
- CE3.** Desenvolvimento de um módulo fuzzy para tratamento de incertezas e valores imprecisos, voltado para critérios críticos como latência, energia, estabilidade do canal e carga computacional. O módulo fuzzy será integrado ao sistema de decisão, auxiliando na suavização das transições e na prevenção de falhas abruptas.
- CE4.** Construção e avaliação de um mecanismo de aprendizado por reforço capaz de identificar padrões dinâmicos e otimizar a ação de descarregamento em tempo real.
- CE5.** Implementação do modelo completo em ambiente ns-3 (versão 3.42), utilizando o 5G-LENA e o suporte ao V2X, permitindo avaliar de forma rigorosa métricas como latência, taxa de sucesso, consumo energético, throughput e eficiência do uso do canal.
- CE6.** Comparação sistemática com métodos de referência da literatura, incluindo heurísticas tradicionais, algoritmos matemáticos e abordagens baseadas exclusivamente em heurísticas, destacando os ganhos obtidos pela integração híbrida ML–Fuzzy.
- CE7.** Produção de um conjunto de diretrizes práticas para pesquisadores e engenheiros interessados em projetar sistemas de task offloading veicular, evidenciando cenários onde o modelo híbrido apresenta maior vantagem.

1.8 Publicação

Como parte do desenvolvimento deste trabalho, foi submetido e aceito um artigo no periódico internacional *Journal of Cloud Computing* (Springer), classificado como Q1 nas bases *Scopus/SJR* e posicionado no percentil superior da área de *Computer Networks and Communications*. O artigo, intitulado “*FOFF: an energy-efficient task offloading in VEC-enabled vehicular networks using fuzzy TOPSIS*” (??), apresenta uma versão preliminar do modelo híbrido

de descarregamento de tarefas proposto nesta dissertação, incluindo a arquitetura conceitual e os primeiros resultados derivados de experimentos em simulação. A publicação serviu tanto como validação da relevância científica da abordagem quanto como etapa intermediária para o refinamento metodológico, a ampliação dos cenários simulados e o aprofundamento das análises apresentadas nos capítulos seguintes.

1.9 Organização do Trabalho

2 FUNDAMENTAÇÃO TEÓRICA

2.0.1 Redes Veiculares

2.0.2 Comunicação 5G e Sidelink

2.0.3 Computação Veicular de Borda

2.0.4 Descarregamento de Tarefas

2.0.5 Requisitos Metodológicos para Simulação Não Estacionária em Redes Veiculares

Os trabalhos que propõem soluções para o descarregamento de tarefas em redes veiculares frequentemente adotam uma premissa simplificadora na modelagem do sistema computacional: a construção do ambiente de simulação baseada em modelos analíticos determinísticos. Um exemplo típico é o cálculo do tempo de processamento definido como a razão entre os ciclos computacionais requeridos e a frequência de Unidade Central de Processamento (CPU) disponível, somado ao emprego de modelos de consumo energético fundamentados na potência dinâmica e em aproximações derivadas (????).

Embora essas escolhas tornem o problema tratável e facilitem a comparação de algoritmos em condições controladas, elas tendem a limitar a fidelidade ao representar o ambiente dinâmico das redes veiculares. Adicionalmente, a derivação direta de uma grandeza a partir de outra cria uma forte correlação entre o tempo de processamento e o consumo energético. Tal situação mostra-se problemática na prática, uma vez que análises estatísticas como regressão linear, Análise de Componentes Principais (PCA) e certos algoritmos de aprendizado requerem matrizes de covariância bem condicionadas, requisito não satisfeito quando há alta colinearidade entre as variáveis preditoras (??). Se a energia consumida for estritamente proporcional ao tempo de processamento, torna-se inviável distinguir o impacto individual de cada variável no desempenho do sistema.

Para ilustrar essa questão, considere o modelo usual para calcular o tempo de processamento (Equação 2.1) e o consumo energético (Equação 2.2) de uma tarefa i em um nó j :

$$T_{i,j}^{\text{proc}} = \frac{C_i}{f_j} \quad (2.1)$$

$$E_{i,j}^{\text{proc}} = \kappa_j f_j^2 C_i \quad (2.2)$$

onde C_i representa o número de ciclos computacionais necessários para processar a tarefa i , f_j é a frequência do processador do nó j , e κ_j é uma constante de consumo energético dependente da capacitância efetiva da arquitetura do nó j . Observe que, a partir da Equação 2.1, obtém-se:

$$C_i = T_{i,j}^{\text{proc}} f_j \quad (2.3)$$

e, ao substituirmos em (2.2), tem-se:

$$E_{i,j}^{\text{proc}} = \kappa_j f_j^2 (T_{i,j}^{\text{proc}} f_j) = \kappa_j f_j^3 T_{i,j}^{\text{proc}} \quad (2.4)$$

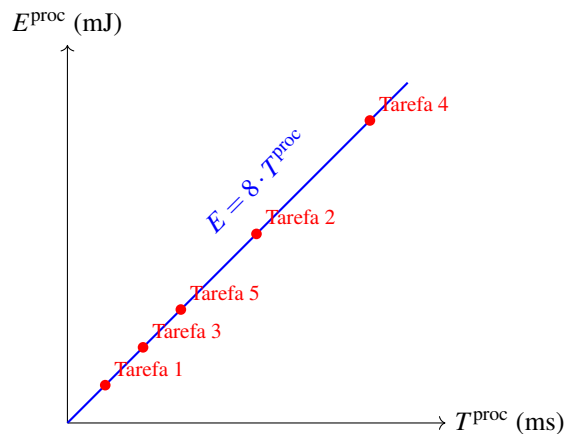
Dessa forma, $E_{i,j}^{\text{proc}}$ torna-se uma função linear de $T_{i,j}^{\text{proc}}$ ao se fixarem os valores de κ_j e f_j , estabelecendo uma correlação perfeita entre as grandezas. A Tabela 1 exemplifica esse comportamento assumindo $\kappa_j = 10^{-27}$ e $f_j = 2$ GHz; a Figura 3 ilustra graficamente esta relação.

Tabela 1 – Correlação entre tempo de processamento e consumo energético em um cenário típico de simulação

Tarefa	C (ciclos)	T^{proc} (ms)	E^{proc} (mJ)
1	2×10^6	1.0	8.0
2	5×10^6	2.5	20.0
3	1×10^6	0.5	4.0
4	8×10^6	4.0	32.0
5	3×10^6	1.5	12.0

Fonte: Elaborado pelo autor

Figura 3 – O gráfico ilustra a colinearidade perfeita: todos os pontos alinham-se em uma reta, indicando que uma variável é redundante em relação à outra.



Fonte: Elaborado pelo autor

A partir dos resultados, infere-se que $E_{i,j}^{\text{proc}} = 8 \times T_{i,j}^{\text{proc}}$. O cálculo da correlação de Pearson entre as duas séries da Tabela 1 resulta em $\widehat{\text{Corr}}(T_{\text{proc}}, E_{\text{proc}}) = 1$, confirmando a colinearidade perfeita. Isso implica que as variáveis carecem de independência estatística, ou seja, apresentam redundância informacional completa. Consequentemente, o uso de modelos de sistema excessivamente simplificados (por exemplo, assumindo κ_j e f_j constantes no tempo) deve ser realizado com cautela, pois pode mascarar *trade-offs* relevantes entre atraso e energia, levando a conclusões imprecisas sobre o desempenho das soluções de decisão de descarregamento de tarefas.

Outra simplificação frequente reside na geração da carga de trabalho. Em muitos ambientes simulados, a chegada de tarefas segue processos de *Poisson*, enquanto atributos como tamanho, densidade computacional e prazos (*deadlines*) são definidos a partir de distribuições uniformes ou normais. Embora útil para testes iniciais, essa prática não descreve adequadamente a variabilidade dos cenários reais, onde a demanda flutua conforme o contexto. Empiricamente, a validade da hipótese de chegadas puramente *Poisson* é contestada em redes de dados reais, falhando em capturar a variabilidade do tráfego em grandes escalas (??).

Formalmente, a execução de uma simulação gera séries temporais que descrevem a evolução das variáveis do sistema (e.g., taxa de chegada, carga do processador). Nesse contexto, uma série é dita **estacionária** quando suas propriedades estatísticas fundamentais, como média e variância, permanecem constantes ao longo do tempo (??). Por outro lado, traços **não estacionários** exibem alterações nessas propriedades, manifestando-se como tendências, mudanças de variância ou alternância entre regimes distintos.

Como resultado, cenários construídos exclusivamente sobre médias e desvios-padrão fixos (ou amostragens i.i.d. sem memória) falham em refletir a complexidade do mundo real. Contudo, em cenários não estacionários, é possível representar fenômenos comuns, como: picos e vales de demanda; degradação variável do canal devido à mobilidade; e flutuações no tempo de processamento causadas por contenção de recursos. Em síntese, a não estacionariedade é um fator que altera fundamentalmente a complexidade do problema, devendo ser um requisito central em avaliações realistas (????).

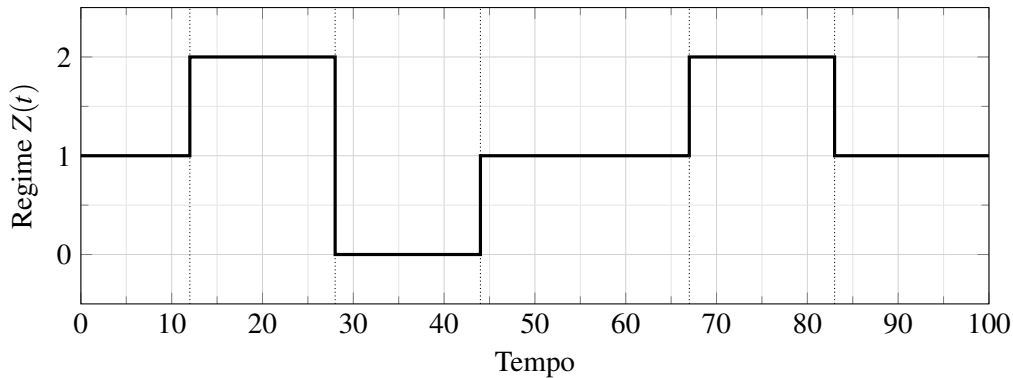
Diante disso, soluções para descarregamento de tarefas deveriam ser avaliadas preferencialmente em ambientes não estacionários, onde as condições variam ao longo do tempo em fases de comportamento persistentes. Uma abordagem controlável para introduzir esse comportamento é empregar modelos com mudança de regime, nos quais um processo de estados discretos governa, por intervalos de tempo, diferentes níveis de carga e condições operacionais

(??????).

Em redes, essa técnica é amplamente utilizada para capturar rajadas (*burstiness*) e correlações temporais em tráfego e qualidade de enlace, por meio de modelos Markov-modulados (p.ex., MMPP e variações do modelo de Gilbert–Elliott) (??????). No contexto de computação de borda, os regimes podem representar, de forma unificada, faixas para parâmetros de tarefa e condições de recursos computacionais e energéticos, evitando que o cenário se reduza a amostragens independentes. A Figura 4 exemplifica uma trajetória de regimes típica.

Figura 4 – Exemplo de trajetória de regimes $Z(t)$ ao longo do tempo (sinal em degraus).

Cada patamar representa um tempo de permanência no regime; as transições representam mudanças de fase (Markov-modulado).



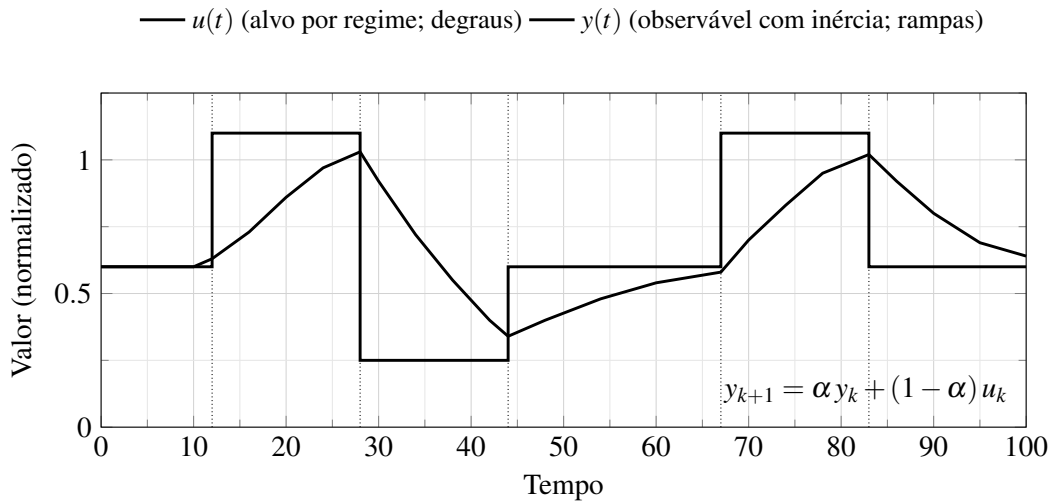
Fonte: Elaborado pelo autor

Esse procedimento também mitiga o problema de colinearidade discutido anteriormente. Ao permitir que $\kappa(t)$ e $f(t)$ evoluam como processos com regimes próprios e escalas de tempo distintas, o consumo energético deixa de ser um simples reescalonamento do tempo de processamento. O simulador passa a produzir um cenário onde tempo e energia são correlacionados mas não redundantes, aproximando-se de situações reais onde o escalonamento do sistema operacional e a heterogeneidade do hardware introduzem variabilidade independente.

Além disso, para evitar mudanças instantâneas irreais nas variáveis físicas, é desejável impor inércia ao sistema por meio de uma dinâmica de primeira ordem. Formalmente, define-se $u(t)$ como o valor alvo imposto pelo regime (sinal em degraus) e $y(t)$ como a resposta observável do sistema. A evolução temporal de $y(t)$ pode ser descrita pela equação de recorrência $y_{k+1} = \alpha y_k + (1 - \alpha)u_k$, onde $\alpha \in [0, 1)$ é o fator de inércia que controla a suavidade da transição, conforme ilustrado na Figura 5. Essa escolha é consistente com a prática de modelagem em arquitetura de computadores, onde a evolução de grandezas agregadas (como a temperatura) é representada por redes equivalentes de resistências e capacitâncias (modelos RC), que induzem

uma resposta suavizada no tempo (??).

Figura 5 – Mesmo regime da Figura 4, agora mapeado para um alvo $u(t)$ (degraus) e para uma variável observável $y(t)$ que responde com inércia de primeira ordem (rampas suaves).



Fonte: Elaborado pelo autor

Por fim, do ponto de vista experimental, essa construção mantém a controlabilidade do cenário — via matrizes de transição e tempos de permanência — ao mesmo tempo que evita a validação em um ambiente estático. Torna-se possível, assim, avaliar políticas de *task offloading* sob mudanças de fase coerentes, alinhando a simulação à dinâmica das redes veiculares.

2.0.6 Perfis de Tarefas no Contexto de Redes Veiculares

PRECISA REVISAR O TEXTO - TOTALMENTE GERADO POR IA

Nesta seção, as tarefas são organizadas em perfis de aplicação para evitar que o gerador de workload produza combinações artificiais de tamanho, densidade computacional e deadline. A motivação prática é que, em cenários de *task offloading* em MEC/VEC, esses parâmetros variam por classe de aplicação e por condições operacionais, e não como amostras totalmente independentes a cada tarefa. Assim, cada perfil define faixas para (i) tamanho da entrada (*size*, em bytes), (ii) densidade computacional (*density*, em ciclos por byte) e (iii) deadline (em segundos). Como parte da consistência de unidades, quando a literatura reporta a complexidade em ciclos por bit, a conversão usada aqui é $\text{cycles/byte} = 8 \cdot \text{cycles/bit}$.

Os valores numéricos adotados nos perfis a seguir devem ser interpretados como

parametrizações de simulação inspiradas por intervalos frequentemente usados em trabalhos de offloading em MEC e IIoT. Esses trabalhos assumem tarefas com tamanhos na ordem de dezenas a centenas de kilobytes, complexidades em ciclos por bit e prazos que variam de regimes sensíveis a atraso até regimes mais tolerantes (?????). Além disso, quando se deseja representar não estacionariedade por mudanças de modo, é natural organizar o workload em regimes persistentes. Esse tipo de organização pode ser formalizado por modelos do tipo semi-Markov, nos quais o tempo de permanência em um estado é tratado explicitamente (??).

O primeiro perfil representa aplicações sensíveis a atraso com interação frequente, nas quais o orçamento de tempo é da ordem de dezenas a poucas centenas de milissegundos. Em estudos de offloading em MEC e IIoT, é comum parametrizar cenários com deadlines curtos para capturar o efeito de fila e contenção de CPU no cumprimento de prazos (????). Para este perfil, adotam-se faixas conservadoras que preservam o caráter *delay-sensitive*: tamanho entre 50 KB e 380 KB, densidade entre 4000 e 8000 ciclos/byte (equivalente a 500 a 1000 ciclos/bit) e deadline fixado em 0,10 s. Esses valores procuram reproduzir um regime em que pequenas variações no workload já têm impacto direto na taxa de cumprimento de deadline, sem exigir hipóteses fortes sobre a distribuição exata das tarefas.

O segundo perfil descreve um cenário de MEC mais geral e heterogêneo, no qual o sistema atende tarefas com tolerância de atraso da ordem de 1 s, com tamanhos e complexidades variando em uma faixa mais ampla. Trabalhos de otimização e estratégia de offloading em MEC usualmente consideram tarefas com complexidades na ordem de centenas a poucos milhares de ciclos por bit e prazos mais relaxados quando o objetivo é estudar trade-offs de energia, utilização de recursos e estabilidade de desempenho (????). Portanto, este perfil utiliza tamanho entre 60 KB e 500 KB, densidade entre 4000 e 12000 ciclos/byte e deadline em 1,0 s. A função desse perfil é oferecer um regime de referência menos restritivo, no qual políticas simples podem ter bom desempenho mesmo sob variações moderadas de carga.

O terceiro perfil concentra-se em conjuntos de tarefas com tamanho médio, com o objetivo de representar cenários em que o volume de dados não é extremo mas a variabilidade de complexidade computacional ainda é relevante. Esse tipo de parametrização é compatível com cenários de IIoT, nos quais aplicações podem alternar entre tarefas com baixo volume de dados e tarefas com maior custo de processamento, dependendo do tipo de medição e do estágio do pipeline (????). Neste caso, adota-se tamanho entre 50 KB e 75 KB, densidade entre 4000 e 12000 ciclos/byte. Para o deadline, quando não há um valor explícito associado a uma aplicação específica no estudo considerado, utiliza-se um valor intermediário como 0,20 s apenas para

manter coerência com o objetivo do simulador, isto é, permitir atrasos perceptíveis sem tornar o sistema trivialmente inviável. Essa escolha deve ser tratada como parâmetro de cenário e não como propriedade universal do tipo de aplicação.

O quarto perfil representa tarefas intensivas em computação com deadline mais relaxado. Em cenários de offloading, esse caso é útil para separar o efeito de volume de dados do efeito de custo computacional, pois o gargalo principal tende a migrar do canal para a CPU, alterando o trade-off entre executar localmente e descarregar para a borda (??). Para este perfil, define-se tamanho fixo em 60 KB, densidade fixa em 253440 ciclos/byte e deadline em 10 s. A intenção é capturar regimes em que a variação de desempenho é dominada pelo custo computacional, o que também favorece a análise de regimes persistentes e mudanças de modo ao longo do tempo (??).

Por fim, o quinto perfil é um perfil de estresse com deadlines muito curtos, usado para testar robustez do simulador e das políticas em situações nas quais o efeito de fila e a disponibilidade de recursos dominam completamente a taxa de sucesso. Esse tipo de caso aparece de forma natural em cenários que combinam comunicação direta (D2D) com MEC, justamente por induzir forte competição por recursos e por exigir decisões rápidas quando o sistema opera próximo ao limite (??). Neste perfil, o tamanho varia entre 62,5 KB e 125 KB, a densidade é 4000 ciclos/byte (500 ciclos/bit) e o deadline é 0,002 s. Esses valores devem ser entendidos como configuração de estresse e não como recomendação geral para todos os cenários.

A Tabela 2 resume os perfis de aplicação adotados no gerador de tarefas, incluindo as faixas de tamanho, densidade computacional e deadline usadas em cada caso. O objetivo dessa organização é manter consistência de unidades e evitar combinações artificiais de parâmetros, preservando classes de aplicações com restrições de tempo distintas, como cenários sensíveis a atraso e cenários mais tolerantes (??????).

Esses perfis devem ser interpretados como parametrizações de simulação inspiradas por hipóteses comuns na literatura e não como valores universais para qualquer sistema MEC/-VEC. Em particular, o perfil de estresse (ID 4) é incluído para avaliar robustez sob deadlines muito curtos, situação em que efeitos de fila e contenção de recursos tendem a dominar o cumprimento de prazos. Essa característica é compatível com cenários de alta competição por recursos discutidos em formulações que integram D2D e MEC (??).

No simulador, cada tarefa é parametrizada por três grandezas: tamanho de entrada (em bytes), densidade computacional (em ciclos por byte) e deadline (em segundos). Como

Tabela 2 – Resumo dos perfis de aplicação adotados para o gerador de tarefas. O tamanho está em KB (decimal), a densidade computacional em ciclos/byte e o deadline em segundos.

ID	Perfil	Size (KB)	Density (ciclos/byte)	Deadline (s)
0	Delay-sensitive / interativo	50–380	4000–8000	0,10
1	MEC geral / tolerante a atraso	60–500	4000–12000	1,0
2	Tamanho médio (workload base)	50–75	4000–12000	0,20
3	Compute-intensivo / deadline relaxado	60 (fixo)	253440 (fixo)	10,0
4	Estresse (ultra baixa latência)	62,5–125	4000 (fixo)	0,002

Notas: (i) Quando a complexidade é reportada em ciclos/bit, a conversão usada é $\text{ciclos/byte} = 8 \cdot \text{ciclos/bit}$. (ii) Os perfis foram inspirados por parametrizações usuais na literatura de offloading em MEC/IIoT e por configurações de estresse para avaliar robustez (???????).

a literatura nem sempre reporta essas grandezas nas mesmas unidades, adotamos conversões explícitas para manter consistência: quando o artigo informa tamanho em *kb* (kilobits), convertemos para bytes por $(kb \cdot 1000)/8$; quando a densidade é dada em ciclos por bit, convertemos para ciclos por byte multiplicando por 8. Esse cuidado evita que diferenças de convenção (por exemplo, bits versus bytes e kilobit versus kilobyte) gerem perfis incoerentes no gerador de tarefas.

Para os perfis de aplicação, usamos como âncora um conjunto típico de parâmetros reportados em estudos de offloading com múltiplos servidores MEC. Em particular, Fang et al. configuram tamanhos de tarefas na faixa de 300 a 1000 kb e densidades de processamento na faixa de 500 a 800 ciclos/bit, valores que se traduzem diretamente para o espaço do simulador após conversão de unidades (??). Esses intervalos são úteis porque produzem variação simultânea de carga de comunicação e carga de computação, o que é desejável quando o objetivo é estudar decisões de offloading sob concorrência e variação temporal.

Para os perfis de CPU, adotamos faixas de capacidade compatíveis com as parametrizações mais comuns em trabalhos de MEC. Fang et al. consideram capacidade de computação do dispositivo (UE) entre 0,5 e 1 GHz e capacidade de servidor MEC em torno de 5 GHz (??). Além disso, Yang et al. utilizam 1 GHz como capacidade local em seus experimentos de alocação eficiente de recursos em MEC (??). Essas escolhas não têm a pretensão de representar um hardware específico mas sim de criar regimes de capacidade suficientemente separados para que a decisão local versus remota tenha consequências mensuráveis em latência e fila.

A Tabela 3 resume os perfis de CPU utilizados no simulador. A intenção é representar classes de capacidade computacional com separação suficiente para que a decisão de execução

local versus offloading tenha efeitos mensuráveis em fila e tempo de serviço, sem comprometer a reprodutibilidade do cenário. Os valores foram escolhidos para permanecer compatíveis com parametrizações típicas encontradas em trabalhos de MEC, que abstraem a computação local e a computação na borda por frequências efetivas (????).

Tabela 3 – Resumo dos perfis de CPU. A frequência é dada em GHz (frequência efetiva usada no modelo), e o número de núcleos representa a capacidade de paralelismo do recurso no simulador.

ID	Perfil	$f_{\min}$ (GHz)	$f_{\max}$ (GHz)	Cores
0	Dispositivo local (UE/veículo), capacidade variável	0,5	1,0	1
1	Servidor de borda (MEC/RSU), capacidade alta	5,0	5,0	1
2	Dispositivo local (UE/veículo), capacidade fixa	1,0	1,0	1

Notas: (i) Os valores refletem parametrizações típicas de simulação em MEC, nas quais a computação é abstraída por uma frequência efetiva e um recurso de servidor (????). (ii) Caso o simulador modele paralelismo por múltiplos *servers*, o campo *Cores* pode ser interpretado como capacidade do recurso (por exemplo, número de slots).

Para perfis de energia de computação, incluímos dois modelos complementares, ambos recorrentes na literatura, justamente para reduzir a colinearidade entre tempo de processamento e energia. O primeiro modelo segue a forma quadrática em frequência típica de abstrações de DVFS, $E = \kappa f^2 b$, em que b é o número de ciclos de CPU e κ é um coeficiente de arquitetura. Essa forma e uma parametrização representativa aparecem no estudo de alocação energética para múltiplas tarefas em MEC (??). O segundo modelo usa energia constante por ciclo, $E = \varepsilon b$, que também é empregado como simplificação prática em experimentos de MEC (??). Ao permitir alternar o regime de CPU e o regime de energia de forma parcialmente independente (por exemplo, mudando κ ou ε sem mudar f), o simulador consegue gerar cenários não estacionários com regimes sem forçar que toda variação temporal de energia seja apenas um reflexo mecânico da variação do tempo de serviço.

A Tabela 4 apresenta os perfis de energia computacional. O objetivo não é reproduzir fielmente uma microarquitetura específica mas oferecer dois modelos recorrentes na literatura: um baseado em dependência quadrática com a frequência (associado a abstrações de DVFS) e outro baseado em energia constante por ciclo. Essa duplicidade é útil para reduzir colinearidade entre tempo de processamento e energia em cenários não estacionários, permitindo que regimes de energia possam variar sem serem apenas um reflexo mecânico do regime de CPU (????).

Tabela 4 – Resumo dos perfis de energia computacional. O perfil 0 usa um modelo dependente de frequência, enquanto o perfil 1 usa energia constante por ciclo, ambos recorrentes na literatura.

ID	Modelo	Parâmetros
0	$E = \kappa f^2 b$ (modelo quadrático em f)	$\kappa = 10^{-28}$; f em Hz; b em ciclos (??)
1	$E = \varepsilon b$ (energia constante por ciclo)	$\varepsilon = 10^{-10}$ J/ciclo; b em ciclos (??)

Notas: (i) b representa o número total de ciclos de CPU exigidos pela tarefa (por exemplo, $b = \text{size} \cdot \text{density}$). (ii) Ao alternar o perfil de energia de forma independente do perfil de CPU, é possível gerar regimes não estacionários sem impor correlação forte e permanente entre tempo de serviço e energia.

2.0.7 Aprendizado por Reforço

Nesta subseção, apresentam-se os conceitos fundamentais sobre aprendizado por reforço (*Reinforcement Learning* – RL), destacando as características que tornam esse paradigma adequado à resolução de problemas com premissas dinâmicas.

Inicialmente, é importante distinguir os conceitos de *algoritmo* e *modelo*. Na computação clássica, um algoritmo consiste em uma sequência finita de instruções lógicas destinadas a converter entradas em uma solução determinada explicitamente. Contudo, no Aprendizado de Máquina (*Machine Learning* - ML), o processo torna-se indutivo, algoritmos são utilizados para processar grandes volumes de dados (*datasets*) e extrair correlações intrínsecas, muitas vezes complexas demais para a análise humana direta.

O produto desse esforço computacional é o *modelo*, que é uma representação das relações aprendidas, capaz de generalizar o conhecimento para inferir respostas a partir de novos dados. Com base na natureza dessa interação, as soluções de ML classificam-se em três paradigmas fundamentais: aprendizado supervisionado, não-supervisionado e aprendizado por reforço.

O paradigma supervisionado assemelha-se intuitivamente ao aprendizado guiado por um “tutor”. Dispõe-se de um conjunto de dados, composto por variáveis preditoras (entradas) e suas respectivas variáveis-alvo (saídas desejadas) fornecidas pela “tutor”, permitindo que o algoritmo aprenda a relação entre entrada e saída.

O objetivo é criar um modelo que minimize a discrepância (erro) entre a resposta predita e a variável-alvo. A Tabela 5 exemplifica um conjunto de dados composto por múltiplos previsores ($x_1, \dots, x_n$) e uma variável-alvo (rótulo) (y).

A relação entre as variáveis preditoras e a variável-alvo pode ser visualizada através da Figura 6 e nela são exibidos os dados e a reta de regressão ajustada. A linha representa a tendência aprendida que minimiza a distância média para os pontos observados. Vale ressaltar

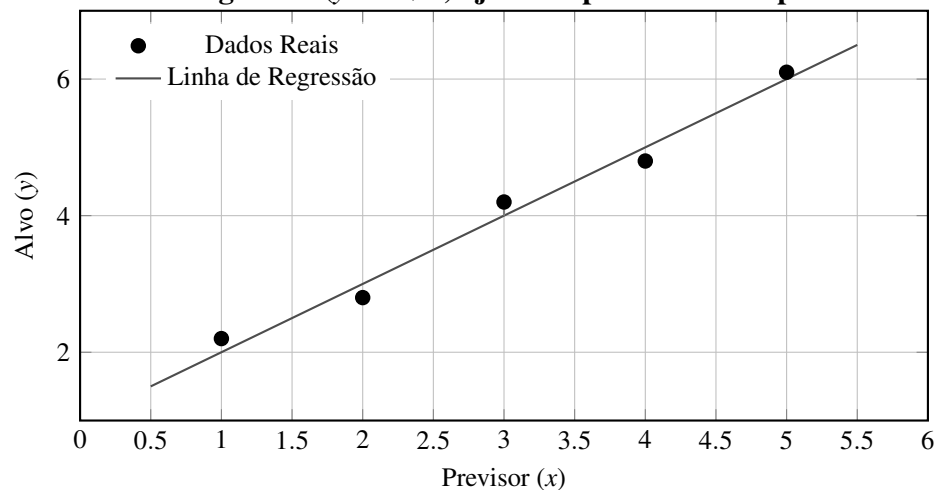
Tabela 5 – Exemplo de conjunto de dados supervisionado com múltiplos previsores e um alvo.

Previsor 1 (x_1)	...	Previsor n (x_n)	Alvo (y)
1.0	...	0.5	2.2
2.0	...	1.2	2.8
3.0	...	1.8	4.2
$\vdots$	$\ddots$	$\vdots$	$\vdots$
5.0	...	3.1	6.1

Fonte: Elaborado pelo autor.

que, para fins de visualização gráfica (Figura 6), utilizou-se apenas uma variável preditora.

Figura 6 – Regressão linear simples: Dados observados (pontos) e reta de regressão ($y \approx x + 1$) ajustada pelo modelo supervisionado.



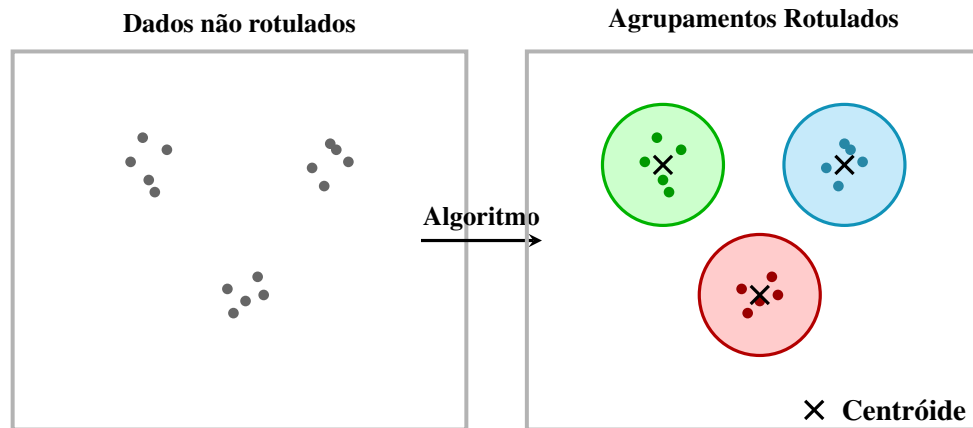
Fonte: Elaborado pelo autor.

No aprendizado não supervisionado, opera-se sobre um conjunto de dados sem rótulos (sem tutor). O objetivo é identificar correlações ou estruturas subjacentes, como o agrupamento de dados semelhantes. Esta abordagem busca descobrir padrões em grandes volumes de dados, como na detecção de anomalias financeiras (??). Na Figura 7, observa-se um conjunto de dados ilustrativo. O objetivo de algoritmos não supervisionados é identificar e delimitar agrupamentos (*clusters*) automaticamente, baseando-se apenas nas características compartilhadas entre as amostras.

Enquanto os paradigmas supervisionado e não supervisionado focam na análise estática de dados — seja para predição ou descoberta de padrões —, o terceiro paradigma aborda o problema da tomada de decisão sequencial.

No aprendizado por reforço clássico, a criação do modelo é realizada de forma diferente, pois não há um *dataset* fixo entregue ao algoritmo. O aprendizado emerge da experiência direta de um *agente* (algoritmo de decisão) com um *ambiente* dinâmico (o mundo e suas

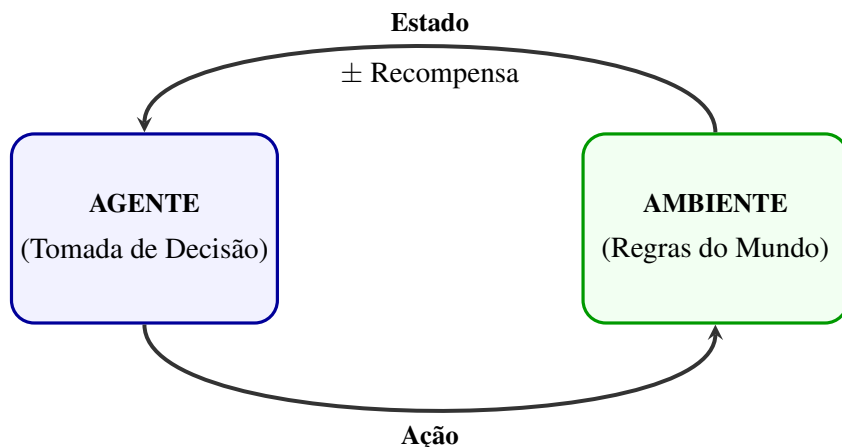
Figura 7 – Conceito de agrupamento (clustering): identificação de estruturas latentes em dados não rotulados.



Fonte: Elaborado pelo autor.

regras). Neste ciclo, o agente observa o estado atual do ambiente, decide uma ação com base em sua estratégia (*política*) e recebe uma resposta na forma de recompensa (ou penalidade). O objetivo final não é apenas acertar um rótulo imediato, mas refinar a política para maximizar as recompensas acumuladas ao longo do tempo (Figura 8).

Figura 8 – Ciclo de interação agente-ambiente no aprendizado por reforço: o agente observa o estado do ambiente, seleciona uma ação segundo sua política aprendida, executa a ação e recebe uma recompensa (consequência), iniciando um novo ciclo.



Fonte: Elaborado pelo autor.

Para fundamentar o funcionamento de decisão no RL, apresenta-se inicialmente o cenário mais elementar: o problema do *k-armed bandit* (k-braços). Neste problema, o agente dispõe de k opções e deve realizar escolhas sequenciais ao longo de um horizonte de tempo finito. Cada opção retorna uma recompensa regida por uma distribuição de probabilidade desconhecida e que não varia ao longo do tempo (*estacionária*). Neste exemplo, ainda não é necessário

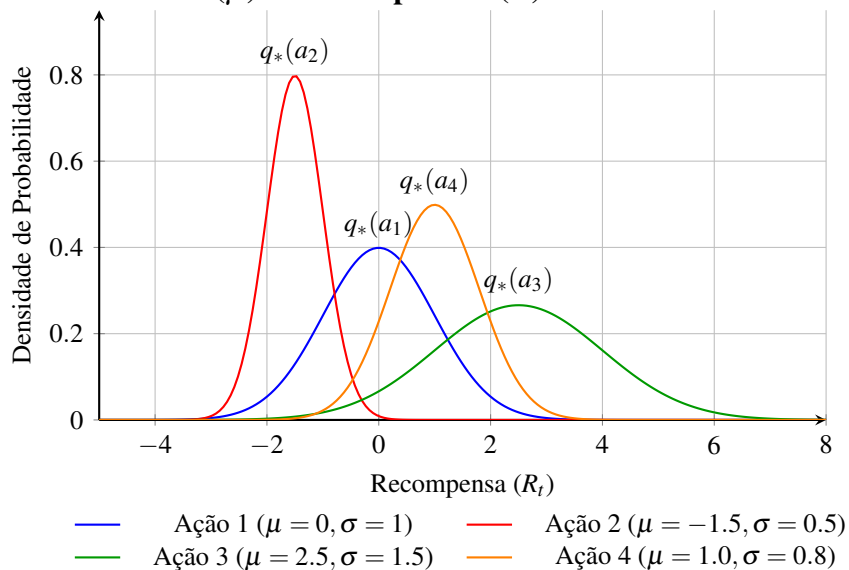
utilizar o conceito do estado pois todas as ações são independentes. O importante é que esse problema permite isolar e analisar exclusivamente o dilema fundamental do paradigma: decidir entre explorar (*explore*) novas opções ou explorar intensivamente (*exploit*) uma opção conhecida vantajosa.

Segundo (??), define-se o valor de uma ação como a recompensa esperada ao escolhê-la. Matematicamente, isso corresponde à esperança condicional da recompensa R_t , dado que a ação a foi selecionada na interação t (Equação 2.5):

$$q_*(a) \doteq \mathbb{E}[R_t | A_t = a] \quad (2.5)$$

Para ilustrar a incerteza inerente ao processo de decisão, a Figura 9 apresenta um exemplo visual de um problema de k -braços com $k = 4$. Cada curva representa a densidade de probabilidade das recompensas associadas a uma ação específica. Observe que, embora a ação a_3 possua o maior valor esperado ($q_*(a_3) = 2.5$), sua alta variância ($\sigma = 1.5$) implica que, em uma única jogada, ela pode retornar uma recompensa inferior à de uma ação subótima, mas mais estável, como a ação a_4 . Cabe ao agente, através de repetidas interações, estimar esses valores esperados para convergir à escolha ótima.

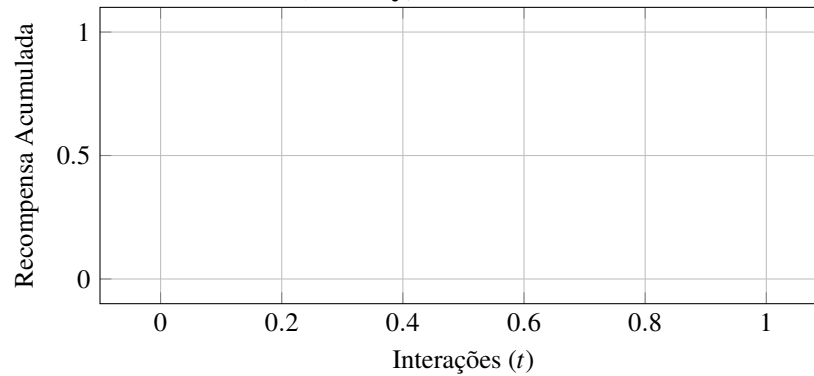
Figura 9 – Distribuições de probabilidade de recompensa para um problema de 4-braços ($k = 4$). Cada curva representa a densidade de probabilidade da recompensa para uma ação específica, ilustrando diferentes médias (μ) e desvios padrão (σ).



Fonte: Elaborado pelo autor.

Como exemplo, considere a evolução da recompensa acumulada apresentada na Figura 10. Esse resultado reflete o comportamento de um agente que, inicialmente desconhecendo as recompensas associadas às ações, experimenta cada opção uma única vez e registra qual delas retornou o maior valor. Suponha que, nessa tentativa preliminar, a ação a_3 tenha se mostrado a mais vantajosa (devido à sua alta variância, que permitiu um retorno alto pontual). Adotando uma estratégia gananciosa, o agente passaria a selecionar a_3 em todas as interações subsequentes. Com isso, perde-se a oportunidade de explorar outras opções que poderiam se revelar superiores a longo prazo — como a ação a_4 , que, apesar de ter retornado menos na primeira tentativa, possui uma média de recompensa robusta e menor variabilidade.

Figura 10 – Recompensa acumulada utilizando a estratégia gananciosa (Greedy).



Fonte: Elaborado pelo autor.

Para lidar com esse dilema, uma estratégia melhor e computacionalmente eficiente, segundo (??), seria estimar o valor de cada ação na interação t utilizando o Algoritmo 1. Neste algoritmo, a ação escolhida seria aquela com a melhor média acumulada até o momento. Para garantir a exploração contínua do espaço de ações, o agente escolhe uma ação aleatória com probabilidade ϵ .

O Algoritmo 1 funciona armazenando o número de vezes que cada ação foi selecionada ($N(a)$) e atualizando a estimativa do valor da ação ($Q(a)$) com base na média incremental das recompensas recebidas. A escolha da ação é feita de forma ϵ -Greedy, com probabilidade $1 - \epsilon$. O agente seleciona a ação com o maior valor estimado, enquanto com probabilidade ϵ , ele escolhe uma ação aleatória, promovendo a exploração.

A Figura 11 ilustra o desempenho de um agente que adota essa abordagem em comparação com outro que utilizada a estratégia gananciosa. Note que, ao longo do tempo, a recompensa acumulada cresce de forma mais consistente (ϵ -Greedy), refletindo uma estratégia mais robusta.

Algoritmo 1: Algoritmo ε -Greedy para o problema k -armed bandit

Require: Número de ações k , valor de exploração ε

Ensure: Estimativas de valor $Q(a)$ para cada ação

1: **Inicializar**, para $a = 1$ até k :

2: $Q(a) \leftarrow 0$

3: $N(a) \leftarrow 0$

4:

5: **Laço:**

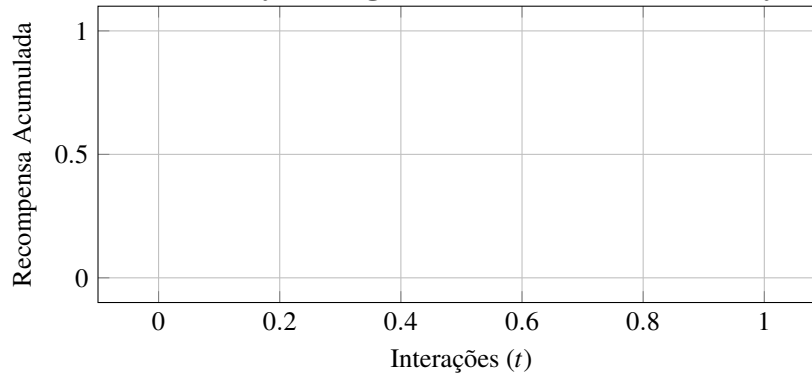
6: $A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{com probabilidade } 1 - \varepsilon \\ \text{uma ação aleatória} & \text{com probabilidade } \varepsilon \end{cases}$

7: $R \leftarrow \text{bandit}(A)$

8: $N(A) \leftarrow N(A) + 1$

9: $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

Figura 11 – Comparação de desempenho: Agente Ganancioso (Greedy) vs. Agente ε -Ganancioso (ε -Greedy).

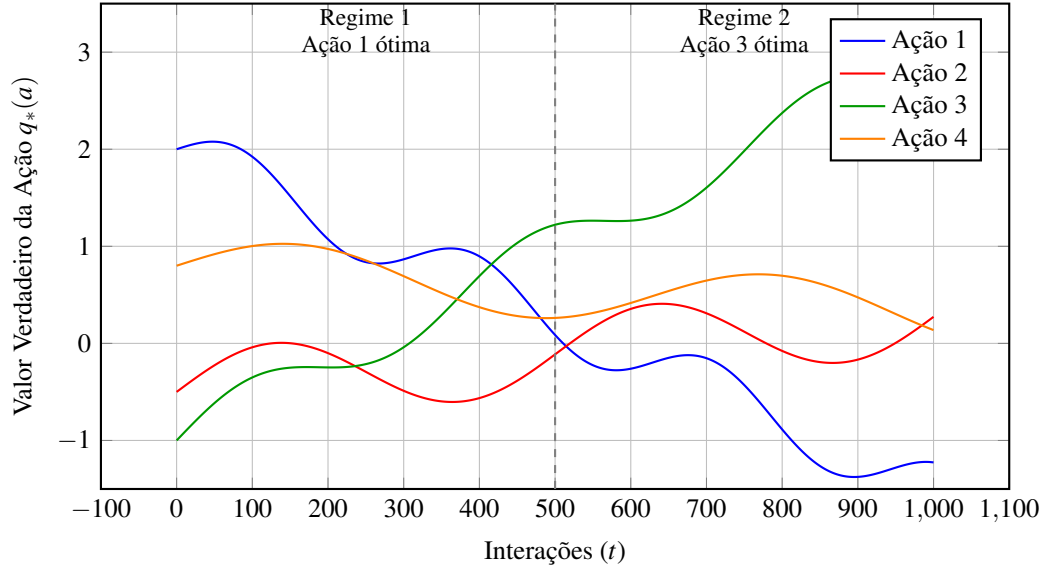


Fonte: Elaborado pelo autor.

Até então não foi necessário lidar com problemas onde as premissas mudam com o tempo, os chamados problemas **não-estacionários**. Em ambientes reais, contudo, as condições podem variar, exigindo que o agente busque estratégias para tratar essas mudanças de comportamento (regime). No contexto do problema k -armed bandit, a não-estacionariedade manifesta-se quando cada k -braço tem sua distribuição de probabilidade de recompensa modificada ao longo do tempo. A Figura 12 ilustra esse cenário, onde as médias das recompensas associadas a cada ação variam temporalmente, evidenciando mudanças de regime que tornam diferentes ações ótimas em diferentes períodos.

Isto induz um novo desafio, o agente deve utilizar estratégias onde a recompensa recebida em uma interação recente possui maior importância do que recompensas obtidas anteriormente. Uma forma simples de implementar isso é substituir a média aritmética por uma atualização com taxa de aprendizado constante (Equação 2.6). Nesta formulação, $\alpha \in (0, 1]$ é a taxa de aprendizado que determina o peso dado às recompensas recentes. Valores maiores

Figura 12 – Exemplo de um problema k -armed bandit não-estacionário: as médias das recompensas ($q_*(a)$) variam ao longo do tempo, caracterizando diferentes regimes onde ações distintas se tornam ótimas. Note como a Ação 3 (verde) inicia inferior mas torna-se dominante após $t \approx 500$, enquanto a Ação 1 (azul) degrada-se.



Fonte: Elaborado pelo autor.

de α fazem o agente reagir mais rapidamente a mudanças, mas aumentam a variabilidade das estimativas.

$$Q_{n+1}(A) = Q_n(A) + \alpha[R_n - Q_n(A)] \quad (2.6)$$

onde $Q_n(A)$ é a estimativa do valor da ação A após ela ter sido selecionada n vezes, e R_n é a recompensa recebida na n -ésima vez que a ação A foi escolhida. O índice n refere-se ao número de vezes que a ação específica A foi selecionada, não ao passo t de tempo global do agente.

O Algoritmo 2 apresenta a versão modificada do Algoritmo 1 para lidar com ambientes não-estacionários, utilizando a atualização exponencialmente ponderada da Equação 2.6.

A Figura 13 ilustra o desempenho comparativo das três estratégias em um ambiente fortemente não-estacionário, onde as médias das recompensas dos braços evoluem continuamente através de um passeio aleatório (*random walk*) aplicado a cada interação, além de mudanças bruscas de regime (*regime shifts*) a cada 1000 interações. Observe que a estratégia com taxa de aprendizado constante ($\alpha = 0.1$) demonstra superior capacidade de adaptação às mudanças, alcançando a maior recompensa acumulada final (aproximadamente 21% superior ao Greedy).

O ambiente não-estacionário é implementado através de dois mecanismos com-

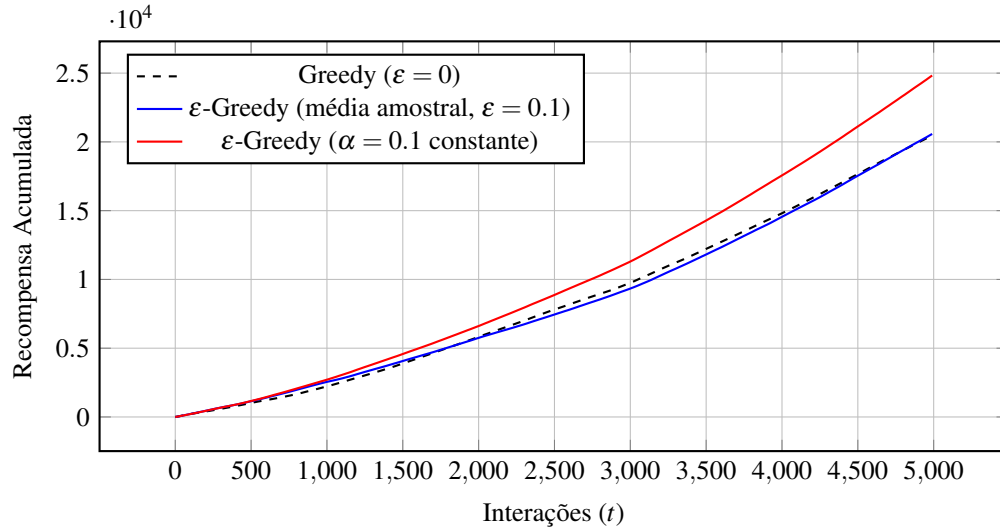
Algoritmo 2: Algoritmo ε -Greedy para o problema k -armed bandit não-estacionário

Require: Número de ações k , valor de exploração ε , taxa de aprendizado α

Ensure: Estimativas de valor $Q(a)$ para cada ação

- 1: **Inicializar**, para $a = 1$ até k :
 - 2: $Q(a) \leftarrow 0$
 - 3:
 - 4: **Laço**:
 - 5: $A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{com probabilidade } 1 - \varepsilon \\ \text{uma ação aleatória} & \text{com probabilidade } \varepsilon \end{cases}$
 - 6: $R \leftarrow \text{bandit}(A)$
 - 7: $Q(A) \leftarrow Q(A) + \alpha[R - Q(A)]$
-

Figura 13 – Comparação de estratégias em ambiente não-estacionário: Greedy vs. ε -Greedy com média amostral vs. ε -Greedy com taxa constante ($\alpha = 0.1$). A estratégia com taxa constante demonstra melhor adaptação às mudanças dinâmicas.



Fonte: Elaborado pelo autor.

plementares: (i) um *random walk* gaussiano contínuo aplicado a cada interação ($\mu_t \leftarrow \mu_{t-1} + \mathcal{N}(0, 0.1^2)$ para cada braço), e (ii) *regime shifts* periódicos a cada 1000 iterações, aplicando perturbações mais bruscas ($\mu_t \leftarrow \mu_{t-1} + \mathcal{N}(0, 2.0^2)$). Os resultados apresentados correspondem à média de 50 execuções independentes com *seeds* distintas (42–91), garantindo robustez estatística dos resultados observados.

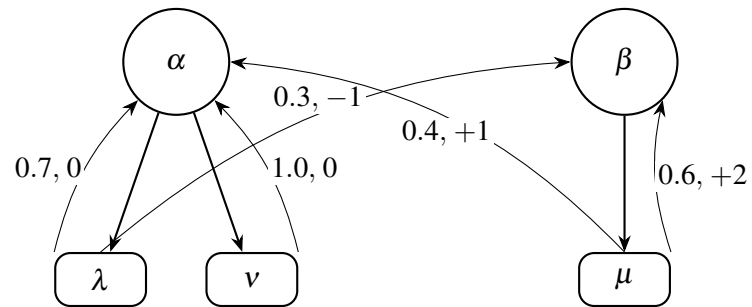


Figura 14 – MDP didático com dois estados (α, β), ações explícitas (λ, v, μ), probabilidades de transição e recompensas associadas.

2.0.8 Lógica Fuzzy

2.0.9 Fuzzy-TOPSIS

2.0.10 Contexto

2.0.11 Considerações Finais

3 METODOLOGIA

A metodologia deste trabalho é fundamentada na simulação, utilizando o simulador de redes NS-3, estendido com os módulos 5G-LENA e C-V2X para modelar com fidelidade o ambiente veicular 5G.

3.1 Modelagem do Sistema e Formulação do Problema

1. Entidades (unidade processadora, tarefa) 1.1 Unidade Processadora (OBU e VEC)

1.1.1 Fila de Processamento (tamanho maximo diferente durante a criação da unidade processadora) 1.1.2 Fila de Transmissão (tamanho maximo diferente durante a criação da unidade processadora) 1.1.3 Frequencia do Clock (GHZ) 1.1.3.1 Modulo para criar perfil heterogeneo de CPU (maquinas lentas, medias, rapidas) 1.1.4 Capacidade de Energia (Joules) 1.1.4.1 Modulo para criar perfil heterogeneo de bateria 1.1.5 Coeficiente de Eficiência Energética 1.1.5.1 Modulo para criar perfil heterogeneo de consumo energetico 1.1.6 Modelagem de Ambientes Não-Estacionários 1.1.6.1 Drift nos Atributos da Unidade Processadora durante a simulação 1.2 Tarefa 1.2.1 Tamanho dos Dados de Entrada (tamanho maximo e minimo dependente do perfil de aplicação - bytes) 1.2.2 Tamanho dos Dados de Saída (tamanho maximo e minimo dependente do perfil de aplicação - bytes) 1.2.3 Atraso Máximo Tolerável (tamanho maximo e minimo dependente do perfil de aplicação - milisegundos) 1.2.4 Densidade de Carga Computacional (tamanho maximo e minimo dependente do perfil de aplicação - ciclos de CPU por byte) 1.2.5 Densidade de Carga Energética (tamanho maximo e minimo dependente do perfil de aplicação - joules por ciclo de CPU) 1.3 Considerações sobre Geração de Tarefas 1.3.1 Distribuição Estatística dos Atributos da Tarefa para diferentes perfis de aplicação 1.3.1.1 Muitos dados e pouca computação 1.3.1.2 Poucos dados e muitas computação 1.3.1.3 Muitos dados e muita computação 1.3.1.4 Poucos dados e pouca computação 1.3.1.5 Media carga de dados e computação 1.3.2 Modelagem de Ambientes Não-Estacionários 1.3.2.1 Drift nos Atributos da Tarefa durante geração 1.3 Considerações sobre Simulador de Eventos Discretos 1.3.1 Adicionar na fila de processamento, remover da fila de processamento 1.3.2 Adicionar na fila de transmissão, remover da fila de transmissão 1.3.3 Iniciar processamento, finalizar processamento 1.3.4 Iniciar transmissão, finalizar transmissão 1.4 Incertezas na Modelagem 1.4.1 Incerteza no tempo de processamento 1.4.2 Incerteza no tempo de transmissão 1.4.3 Incerteza no consumo energético 1.4 Protocolo de Offloading, na camada de aplicação 1.4.1 Pedido de Offload 1.4.2 Resposta ao Pedido (Aceite/Rejeição) 1.4.2.1 Rejeição por fila cheia 1.4.2.2 Rejeição por energia insuficiente

1.4.2.3 Rejeição por não cumprimento do deadline 1.4.3 Envio dos Dados da Tarefa 1.4.4 Retorno do Resultado 1.4.5 Confirmação de Recebimento do Resultado 1.4.6 Timeouts e Retransmissões 1.4.7 Medição de Métricas de Desempenho (latência, consumo energético, taxa de falhas, tamanho média das filas) 1.4.8 Considerações sobre Simulador de Eventos Discretos 1.4.8.1 Enviar pedido de offload 1.4.8.2 Aguardar resposta 1.5 Protocolo de Descoberta de Vizinhaça, na camada de aplicação 1.5.1 Emissão de Beacons Periódicos (via 5G NR e V2X Sidelink) 1.5.1.1 Conteúdo do Beacon (Device Specs, Estado Atual, velocidade, posição GPS, é VEC?, é OBU?, etc) 1.5.2 Tabela de Vizinhos (Neighbor Table) 1.5.2.1 Cálculo do Tempo de Vida Estimado do Enlace 1.5.2.3 Atualização Periódica da Tabela de Vizinhos 1.5.2.4 SNR, RSSI, CQI dos Vizinhos 2. Canal de comunicação 2.1 Modelo de Canal 5G NR 2.1.1 Taxa de Transmissão (Shannon-Hartley) 2.1.2 Modelagem de Interferência 2.1.3 Mobilidade Veicular e Variação do Canal 2.2 Modelo de Canal V2X Sidelink (PC5) 2.2.1 Taxa de Transmissão (Shannon-Hartley) 2.2.2 Modelagem de Interferência 2.2.3 Mobilidade Veicular e Variação do Canal 2.3 Modelagem de Ambientes Não-Estacionários 2.3.1 Drift nos Atributos do Canal durante a simulação 3. Task Offloading 3.1 Modos de Execução 3.1.1 Processamento Local 3.1.2 Offloading V2I (para RSU via 5G NR) 3.1.3 Offloading V2V (para veículo vizinho via Sidelink) 3.2 4. Coleta de métricas de desempenho do task offloading 4.1 Métricas Globais 4.1.1 Atraso Total do Sistema 4.1.2 Consumo Energético Total do Sistema 4.1.3 Índice de Justiça de Jain 4.1.4 Taxa de Sucesso no Offloading 4.1.5 Total de Tarefas Processadas Localmente 4.1.6 Total de Tarefas Offloadadas via V2I 4.1.7 Total de Tarefas Offloadadas via V2V 4.2 Métricas por Política de Offloading 4.2.1 Atraso Médio por Política 4.2.2 Consumo Energético Médio por Política 4.2.3 Taxa de Sucesso no Offloading por Política 4.3.4 Tamanho Médio das Filas por Política 4.3.5 Índice de justiça de Jain por Política 4.3 Análise de Sensibilidade 4.3.1 Impacto dos Pesos na Função Objetivo 4.3.2 Trade-off entre Latência, Consumo Energético e Justiça 4.4 Função Objetivo Global 4.4.1 Minimização do Atraso Total considerando incerteza 4.4.2 Minimização do Consumo Energético Total considerando incerteza 4.4.3 Maximização do Índice de Justiça de Jain 5. Integração com o SUMO 5.1 Geração de Cenários Veiculares Realistas 5.1.1 Cenário urbano com tráfego intenso, tráfego moderado, tráfego leve 5.2 Execução da simulação utilizando o SUMO para mobilidade veicular 6. Políticas de Decisão de Offloading 6.1 Políticas Heurísticas 6.1.1 Processamento Local 6.1.2 Offloading Aleatório 6.1.3 Offloading Ganancioso 6.1.3.1 Baseado na distancia ao VEC 6.1.3.2 Baseado na qualidade do canal 6.1.3.3 Baseado na latência 6.1.3.4 Baseado no consumo energético 6.1.3.5 Baseado na frequencia do clock do VEC 6.1.3.6 Baseado no tempo de vida do enlaces 6.1.3.7 Baseado na função objetivo 6. Criação de testes unitários para

validação dos módulos desenvolvidos 6.1 Testes para o módulo de geração de tarefas 6.2 Testes para o módulo de modelagem do canal 6.3 Testes para o módulo de offloading 6.4 Testes para o protocolo de descoberta de vizinhança 6.5 Testes para coleta de métricas de desempenho do task offloading 7. Usar docker para facilitar a reprodução dos experimentos 8. Usar script em bash e python para automatizar a execução dos experimentos e análise dos resultados 9. Criar logs para facilitar o rastreamento da execução offloading (fim a fim) 10. Documentar o código desenvolvido utilizando Doxygen 11. Tratar exceções e erros durante a simulação para garantir robustez do sistema 11.1 Não confiar 100 por cento nas entradas das funções, tratar todas as exceções possíveis

REFERÊNCIAS

3GPP. *Release 17 Overview*. [S.l.], 2022. Accessed: 2025-01-01.

AHMED, M. *et al.* A survey on vehicular task offloading: Classification, issues and future challenges. **Journal of King Saud University – Computer and Information Sciences**, 2022. Elsevier / ScienceDirect.

AHMED, M.; RAZA, S.; MIRZA, M. A.; AZIZ, A.; KHAN, M. A.; KHAN, W. U.; LI, J.; HAN, Z. A survey on vehicular task offloading: Classification, issues, and challenges. **Journal of King Saud University - Computer and Information Sciences**, v. 34, n. 7, p. 4135–4162, 2022. ISSN 1319-1578. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1319157822001653>.

AKBARI, A. *et al.* DRL-OS: Deep Reinforcement Learning-Based Offloading Strategy to Reduce Task Completion Time in Mobile Edge Computing. **Sensors**, v. 22, n. 23, p. 9212, 2022.

DENG, Q.; MEI, G. Combining self-organizing map and k-means clustering for detecting fraudulent financial statements. In: **2009 IEEE International Conference on Granular Computing**. Nanchang, China: [s.n.], 2009. p. 126–131.

ELLIOTT, E. O. Estimates of error rates for codes on burst-noise channels. **The Bell System Technical Journal**, v. 42, n. 5, p. 1977–1997, 1963.

FANG, J.; SHI, J.; LU, S.; ZHANG, M. An efficient computation offloading strategy with mobile edge computing for iot. **Micromachines**, v. 12, n. 2, p. 204, 2021.

FISCHER, W.; MEIER-HELLSTERN, K. The markov-modulated poisson process (mmpp) cookbook. **Performance Evaluation**, v. 18, n. 2, p. 149–171, 1993.

HAMILTON, J. D. A new approach to the economic analysis of nonstationary time series and the business cycle. **Econometrica**, v. 57, n. 2, p. 357–384, 1989.

HAO, X.; ZHAO, R.; YANG, T.; SCHMEINK, A. A Risk-Sensitive Task Offloading Strategy for Edge Computing in Industrial Internet of Things. **EURASIP Journal on Wireless Communications and Networking**, v. 2021, n. 1, p. 39, 2021.

HE, C.; WANG, W.; JIANG, W.; HE, Z.; WANG, J.; XIE, X. Security for the internet of vehicles with integration of sensing, communication, computing, and intelligence: A comprehensive survey. **Sensors**, v. 25, n. 16, 2025. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/25/16/5119>.

HE, Y.; REN, J.; YU, G.; CAI, Y. D2D Communications Meet Mobile Edge Computing for Enhanced Computation Capacity in Cellular Networks. **IEEE Transactions on Wireless Communications**, v. 18, n. 3, p. 1750–1763, 2019.

HEFFES, H.; LUCANTONI, D. M. A markov modulated characterization of packetized voice and data traffic and related statistical multiplexer performance. **IEEE Journal on Selected Areas in Communications**, v. 4, n. 6, p. 856–868, 1986.

HUANG, X.; YE, Q.; LI, J.; WU, Y. Deep reinforcement learning for offloading and resource allocation in vehicular edge computing. **IEEE Transactions on Vehicular Technology**, v. 69, n. 8, p. 8334–8345, 2020.

HYNDMAN, R. J.; ATHANASOPOULOS, G. **Forecasting: Principles and Practice**. 3. ed. [S.l.]: OTexts, 2021. Online book.

KESHARI, N.; SINGH, D.; MAURYA, A. K. A survey on vehicular fog computing: Current state-of-the-art and future directions. **Vehicular Communications**, v. 38, p. 100512, 2022. Disponível em: <https://doi.org/10.1016/j.vehcom.2022.100512>.

KHAN, P. W.; ABBAS, K.; SHAIBA, H.; MUTHANNA, A.; ABUARQOUB, A.; KHAYYAT, M. Energy efficient computation offloading mechanism in multi-server mobile edge computing—an integer linear optimization approach. **Electronics**, MDPI, v. 9, n. 6, p. 1010, 2020.

KISHOR, A.; CHAKARBARTY, C. Task offloading in fog computing for using smart ant colony optimization. **Wireless personal communications**, Springer, v. 127, n. 2, p. 1683–1704, 2022.

LI, Z.; ZHU, Q. Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing. **Information**, MDPI, v. 11, n. 2, p. 83, 2020.

LIMNIOS, N.; OPRISAN, G. **Semi-Markov Processes and Reliability**. [S.l.]: Birkhäuser Boston, 2001. ISBN 978-0-8176-4196-2.

LIU, J.; LIU, X. Energy-efficient allocation for multiple tasks in mobile edge computing. **Journal of Cloud Computing**, v. 11, p. 71, 2022.

LIU, L.; CHEN, C.; PEI, Q.; MAHARJAN, S.; ZHANG, Y. Vehicular edge computing and networking: A survey. **Mobile Networks and Applications**, v. 26, n. 3, p. 1145–1168, jun. 2021. ISSN 1572-8153. Disponível em: <https://doi.org/10.1007/s11036-020-01624-1>.

LUO, Q.; LUAN, T. H.; SHI, W.; FAN, P. Heterogeneous task oriented data scheduling in vehicular edge computing via deep reinforcement learning. **IEEE Transactions on Vehicular Technology**, v. 73, n. 12, p. 19582–19596, 2024.

MAO, Y.; YOU, C.; ZHANG, J.; HUANG, K.; LETAIEF, K. B. A survey on mobile edge computing: The communication perspective. **IEEE Communications Surveys & Tutorials**, v. 19, n. 4, p. 2322–2358, 2017.

MAO, Y.; ZHANG, J.; LETAIEF, K. B. **Dynamic Computation Offloading for Mobile-Edge Computing with Energy Harvesting Devices**. 2016. ArXiv:1605.05488.

MONTGOMERY, D. C.; PECK, E. A.; VINING, G. G. **Introduction to Linear Regression Analysis**. 6th. ed. Hoboken, NJ: John Wiley & Sons, 2021. Ver Capítulo 9: Multicollinearity. ISBN 978-1-119-57872-7.

NIETO, G.; IGLESIA, I. de la; LOPEZ-NOVOA, U.; PERFECTO, C. Deep reinforcement learning techniques for dynamic task offloading in the 5g edge-cloud continuum. **Journal of Cloud Computing**, v. 13, n. 1, p. 94, 2024. ISSN 2192-113X. Disponível em: <https://doi.org/10.1186/s13677-024-00658-0>.

PAXSON, V.; FLOYD, S. Wide area traffic: The failure of poisson modeling. **IEEE/ACM Transactions on Networking**, IEEE, v. 3, n. 3, p. 226–244, 1995. Prova empírica fundamental de que o tráfego de redes reais viola a suposição de Poisson devido a rajadas (burstiness).

RABAEY, J. M.; CHANDRAKASAN, A. P.; NIKOLIĆ, B. **Digital Integrated Circuits: A Design Perspective**. [S.l.]: Prentice Hall, 2003.

RANI, P.; SHARMA, R. Intelligent transportation system for internet of vehicles based vehicular networks for smart cities. **Computers and Electrical Engineering**, v. 105, p. 108543, 2023. ISSN 0045-7906. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0045790622007583>.

SHI, W.; CHEN, L.; ZHU, X. Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach. **Sensors**, v. 23, n. 17, 2023. ISSN 1424-8220. Disponível em: <https://www.mdpi.com/1424-8220/23/17/7595>.

SKADRON, K.; STAN, M. R.; HUANG, W. *et al.* Temperature-aware microarchitecture. In: **Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)**. [S.l.: s.n.], 2003.

STUDY on channel model for frequencies from 0.5 to 100 GHz (3GPP TR 38.901 version 16.1.0 Release 16). [S.l.], 2020. Release 16.

SUTTON, R. S.; BARTO, A. G. **Reinforcement Learning: An Introduction**. 2nd. ed. Cambridge, MA: MIT Press, 2018. Citation key matching text request; likely refers to Sutton & Barto 2018.

VIEIRA, A. S. de S.; SOUZA, A. B. de; JÚNIOR, J. C. Foff: an energy-efficient task offloading in vec-enabled vehicular networks using fuzzy topsis. **Cluster Computing**, Springer, v. 28, n. 13, p. 822, 2025.

WANG, F.; XU, J.; WANG, X.; CUI, S. Joint offloading and computing optimization in wireless powered mobile-edge computing systems. In: **IEEE International Conference on Communications (ICC)**. [S.l.: s.n.], 2017. p. 1–6.

WANG, M.; YI, H.; JIANG, F.; LIN, L.; GAO, M. Review on offloading of vehicle edge computing. **Journal of Artificial Intelligence and Technology**, v. 2, n. 4, p. 132–143, 2022. Disponível em: <https://doi.org/10.37965/jait.2022.0120>.

WHAIDUZZAMAN, M.; SOOKHAK, M.; GANI, A.; BUYYA, R. A survey on vehicular cloud computing. **Journal of Network and Computer Applications**, v. 40, p. 325–344, 2014. Disponível em: <https://doi.org/10.1016/j.jnca.2013.08.004>.

YANG, H.; KUMARI, P.; ZHANG, Z. *et al.* Industrial Internet of Things in China: A Review. **IEEE Network**, v. 33, n. 5, p. 8–15, 2019.

YANG, Z.; HOU, J.; SHIKH-BAHAEI, M. Energy efficient resource allocation for mobile-edge computation networks with noma. 2019.

YOU, Q.; TANG, B. Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. **Journal of Cloud Computing**, Springer, v. 10, n. 1, p. 41, 2021.

ZHANG, K.; MAO, Y.; LENG, S.; HE, L.; ZHANG, Y.; PENG, X.; VINEL, A. Energy-efficient offloading for mobile edge computing in 5g networks. **IEEE Wireless Communications**, v. 25, n. 2, p. 20–27, 2018.

ZHENG, K.; ZHENG, Q.; CHATZIMISIOS, P.; XIANG, W.; ZHOU, Y. Heterogeneous vehicular networking: A survey on architecture, challenges, and solutions. **IEEE Communications Surveys & Tutorials**, v. 17, n. 4, p. 2377–2396, 2015.

ZHOU, Z.; CHEN, X.; LI, E.; ZENG, L.; LUO, K.; ZHANG, J. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. **Proceedings of the IEEE**, v. 107, n. 8, p. 1738–1762, 2019.

APÊNDICE A – LOREM IPSUM

Suspendisse eget dignissim felis, a faucibus velit. Nunc ornare ullamcorper libero, nec placerat sapien ultricies sed. Quisque consequat metus finibus hendrerit iaculis. Pellentesque in eleifend nulla. Nulla facilisi. Mauris lobortis augue justo, eget hendrerit erat fringilla nec. Cras at nisl et massa sollicitudin tempus. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aliquam a justo libero. Nam suscipit erat feugiat sodales ultrices. Duis eget arcu at justo dapibus tempus eget eu enim. Mauris at lacus tortor.

APÊNDICE B – MODELO DE CAPA

Nunc mauris eros, convallis eget blandit non, ultrices vel sem. Donec hendrerit nibh non turpis dictum, at imperdiet lectus vehicula. Donec maximus risus ac leo viverra, in viverra arcu venenatis. Fusce imperdiet tortor nec lorem pharetra ornare. Morbi a sagittis risus. Suspendisse eu felis blandit ipsum bibendum semper varius eget eros. Sed et magna gravida, pharetra dui eu, eleifend neque. Maecenas nunc magna, rhoncus eget urna non, cursus fermentum risus. In blandit justo aliquam, cursus arcu id, maximus magna. Nulla sit amet vehicula velit, sed ultrices turpis. Vivamus vestibulum ante vitae ipsum vestibulum sagittis.

APÊNDICE C – TERMO DE FIEL DEPOSITÁRIO

Pesquisa: ANÁLISE DA MORTALIDADE INFANTIL COM MALFORMAÇÕES CONGÊNITAS.

Pelo presente instrumento que atende às exigências legais, a Sra. Maria Consuelo Martins Saraiva, “fiel depositário” com o cargo de Secretária Municipal de Saúde de Iracema, após ter tomado conhecimento do protocolo de pesquisa intitulado: ANÁLISE DA MORTALIDADE INFANTIL COM MALFORMAÇÕES CONGÊNITAS. Analisando a repercussão desse estudo no contexto da saúde pública e epidemiologia, autoriza Karla Maria da Silva Lima, enfermeira, aluna do Curso de Mestrado Acadêmico em Enfermagem da Universidade Estadual do Ceará (UECE), sob orientação do Prof. Dr. José Maria de Castro, da UECE, ter acesso aos bancos de dados do Sistema de Informação sobre Nascidos Vivos e do Sistema de Informação sobre Mortalidade da Secretaria Municipal de Saúde de Iracema, objeto deste estudo, e que se encontram sob sua total responsabilidade. Fica claro que o Fiel Depositário pode a qualquer momento retirar sua AUTORIZAÇÃO e ciente de que todas as informações prestadas tornar-se-ão confidenciais e guardadas por força de sigilo profissional, assegurando que os dados obtidos da pesquisa serão somente utilizados para estudo.

ANEXO A – EJEMPLO DE ANEXO

Proin tempus turpis id justo sagittis porta a vel dui. Cras id dictum velit. Vivamus a eleifend lectus, non faucibus tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum sagittis dapibus tortor, in pellentesque quam porttitor eget. Nunc vel feugiat velit. Sed varius nisl lectus, at pulvinar tellus fermentum eget. Vestibulum euismod volutpat nisi, in fringilla sapien mollis et. Nam laoreet, ante vitae euismod viverra, erat odio hendrerit purus, eu pulvinar ligula sapien vitae felis.

ANEXO B – DINÂMICA DAS CLASSES SOCIAIS

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Maecenas vehicula arcu vitae risus varius tincidunt. Phasellus iaculis ultrices risus sit amet cursus. Quisque iaculis ex id iaculis commodo. Phasellus fringilla efficitur orci, ut maximus libero vulputate non. Maecenas fermentum, tortor quis feugiat ultrices, dolor diam molestie felis, sed porttitor tellus dolor ac leo. Vivamus tincidunt dapibus viverra. Duis vulputate pretium tortor ac rhoncus. Quisque id porta dolor. Vivamus sed dignissim quam. Nullam blandit velit et dignissim volutpat. Duis lobortis libero ut risus cursus, id ornare mauris blandit. Sed elementum vel lectus et rutrum. Quisque tempus sodales massa, sed tempor ante. Aenean in ultrices felis.

ÍNDICE

AAA, 45